

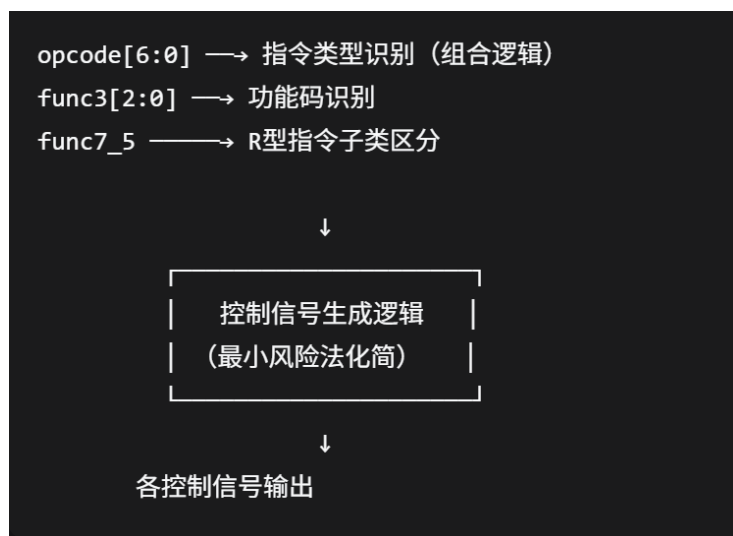
第六次实验报告

251220033 吴淇

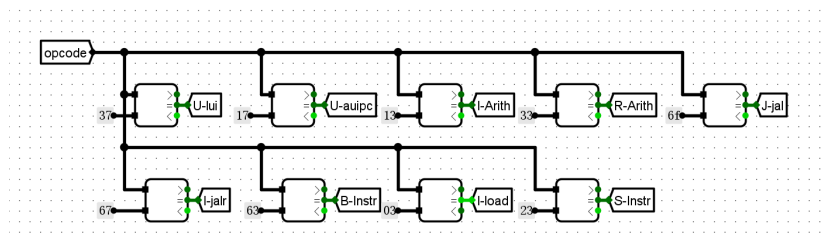
2026 年 5 月 16 日

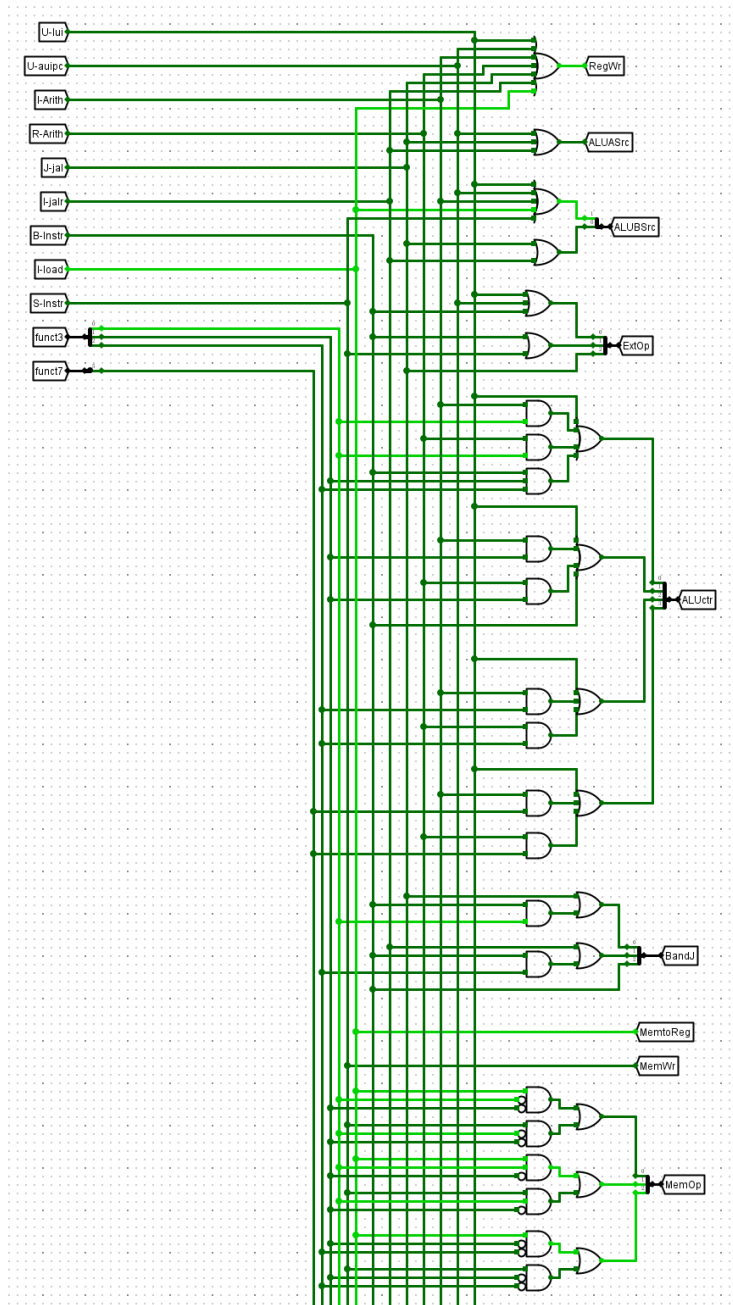
1 实验一 控制器设置实验

1.1 原理图



1.2 电路图





1.3 电路功能

控制信号生产部件根据指令代码中的操作码 opcode、功能码 func3 和功能码 func7 来生成对应的控制信号的。

需要生成的控制信号包括：

ExtOp: 立即数产生器的控制信号，根据指令类型赋值，宽度为 3 位。

ALUASrc: ALU 输入端 A 的选择信号，0: R[Rs1], 1: PC。

ALUBSrc: ALU 输入端 B 的选择信号，宽度为 2 位，00: R[Rs2], 01: 4, 10: imm。

ALUctr: ALU 控制信号，根据指令操作码和功能码赋值，宽度为 4 位。

MemOp: 数据存储器读写操作以及字节长度控制信号，宽度为 3 位。

RegWr: 寄存器堆写使能控制信号，根据指令类型在相应的执行阶段来赋值，高电平有效。

MemWr: 数据存储器写使能控制信号，高电平有效。

BandJ: 用于分支跳转控制器的输入，生成下地址逻辑部件中控制信号 NPCASrc 和 NPCBSrc，宽度为 3 位。

MemtoReg: 寄存器堆写入数据来源控制信号，0: ALU 运算结果，1: 数据存储器读取输出。

NPCASrc: 下地址逻辑中加法器输入端 A 的选择信号，0: PC, 1: R[Rs1]。

NPCBSrc: 下地址逻辑中加法器输入端 B 的选择信号，0: 4, 1: imm。

表 1: RV32I 指令控制信号列表

指令	类型	op[6:0]	func3	func7[5]	ExtOp	RegWr	ALUASrc	ALUBSrc	ALUctrl
lui	U	0110111	×	×	001	1	×	10	1111
auipc	U	0010111	×	×	001	1	1	10	0000
addi	I	0010011	000	×	000	1	0	10	0000
slti	I	0010011	010	×	000	1	0	10	0010
sltiu	I	0010011	011	×	000	1	0	10	0011
xori	I	0010011	100	×	000	1	0	10	0100
ori	I	0010011	110	×	000	1	0	10	0110
andi	I	0010011	111	×	000	1	0	10	0111
slli	I	0010011	001	0	000	1	0	10	0001
srli	I	0010011	101	0	000	1	0	10	0101

续下页

续表：RV32I 指令控制信号列表

指令	类型	op[6:0]	func3	func7[5]	ExtOp	RegWr	ALUASrc	ALUBSrc	ALUctrl
srai	I	0010011	101	1	000	1	0	10	1101
add	R	0110011	000	0	×	1	0	00	0000
sub	R	0110011	000	1	×	1	0	00	1000
sll	R	0110011	001	0	×	1	0	00	0001
slt	R	0110011	010	0	×	1	0	00	0010
sltu	R	0110011	011	0	×	1	0	00	0011
xor	R	0110011	100	0	×	1	0	00	0100
srl	R	0110011	101	0	×	1	0	00	0101
sra	R	0110011	101	1	×	1	0	00	1101
or	R	0110011	110	0	×	1	0	00	0110
and	R	0110011	111	0	×	1	0	00	0111
jal	J	1101111	×	×	100	1	1	01	0000
jalr	I	1100111	000	×	000	1	1	01	0000
beq	B	1100011	000	×	010	0	0	00	0010
bne	B	1100011	001	×	010	0	0	00	0010
blt	B	1100011	100	×	010	0	0	00	0010
bge	B	1100011	101	×	010	0	0	00	0010
bltu	B	1100011	110	×	010	0	0	00	0011
bgeu	B	1100011	111	×	010	0	0	00	0011
lb	I	0000011	000	×	000	1	0	10	0000
lh	I	0000011	001	×	000	1	0	10	0000
lw	I	0000011	010	×	000	1	0	10	0000
lbu	I	0000011	100	×	000	1	0	10	0000
lhu	I	0000011	101	×	000	1	0	10	0000
sb	S	0100011	000	×	010	0	0	10	0000
sh	S	0100011	001	×	010	0	0	10	0000
sw	S	0100011	010	×	010	0	0	10	0000

表 2: RV32I 指令控制信号列表（续）

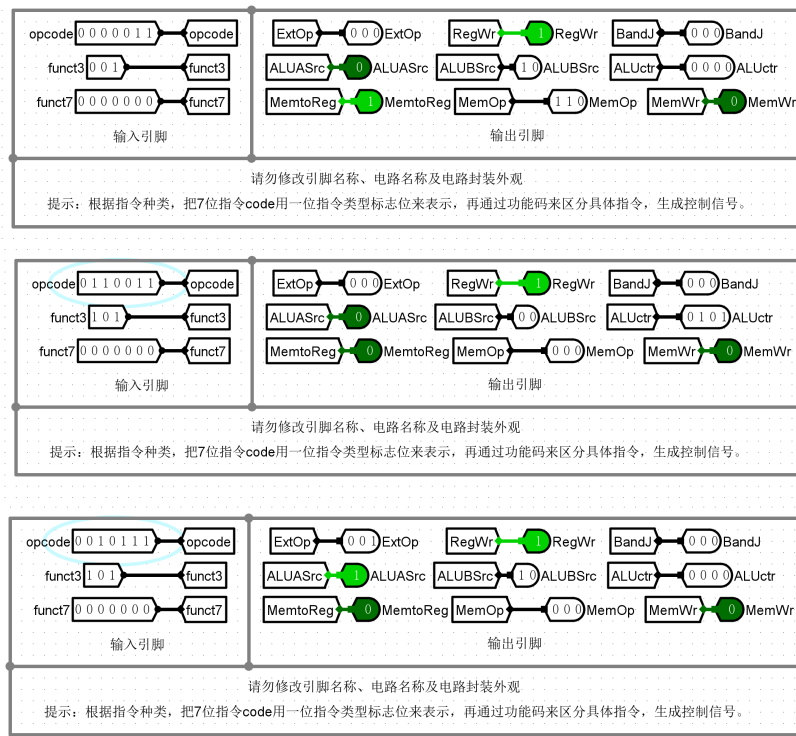
指令	类型	op[6:0]	func3	func7[5]	BandJ	MemtoReg	MemWr	MemOp
lui	U	0110111	×	×	000	0	0	×
auipc	U	0010111	×	×	000	0	0	×
addi	I	0010011	000	×	000	0	0	×
slti	I	0010011	010	×	000	0	0	×
sltiu	I	0010011	011	×	000	0	0	×
xori	I	0010011	100	×	000	0	0	×
ori	I	0010011	110	×	000	0	0	×
andi	I	0010011	111	×	000	0	0	×
slli	I	0010011	001	0	000	0	0	×
srli	I	0010011	101	0	000	0	0	×
srai	I	0010011	101	1	000	0	0	×
add	R	0110011	000	0	000	0	0	×
sub	R	0110011	000	1	000	0	0	×
sll	R	0110011	001	0	000	0	0	×
slt	R	0110011	010	0	000	0	0	×
sltu	R	0110011	011	0	000	0	0	×
xor	R	0110011	100	0	000	0	0	×
srl	R	0110011	101	0	000	0	0	×
sra	R	0110011	101	1	000	0	0	×
or	R	0110011	110	0	000	0	0	×
and	R	0110011	111	0	000	0	0	×
jal	J	1101111	×	×	001	0	0	×
jalr	I	1100111	000	×	010	0	0	×
beq	B	1100011	000	×	100	×	0	×
bne	B	1100011	001	×	101	×	0	×
blt	B	1100011	100	×	110	×	0	×
bge	B	1100011	101	×	111	×	0	×
bltu	B	1100011	110	×	110	×	0	×
bgeu	B	1100011	111	×	111	×	0	×

续下页

续表：RV32I 指令控制信号列表（续）

指令	类型	op[6:0]	func3	func7[5]	BandJ	MemtoReg	MemWr	MemOp
lb	I	0000011	000	×	000	1	0	101
lh	I	0000011	001	×	000	1	0	110
lw	I	0000011	010	×	000	1	0	000
lbu	I	0000011	100	×	000	1	0	001
lhu	I	0000011	101	×	000	1	0	010
sb	S	0100011	000	×	000	×	1	101
sh	S	0100011	001	×	000	×	1	110
sw	S	0100011	010	×	000	×	1	000

1.4 仿真模拟



1.5 错误现象及分析

在完成本次实验的过程中，没有遇到任何问题。


```

test_3:          # sub test
    lui x1, 0x80000
    lui x2, 0xffff8
    sub x15, x1, x2

test_4:          # slt test
    addi x1, x0, 0x7ff
    addi x2, x0, 0x800
    slt x16, x1, x2
    slt x17, x2, x1

test_5:          # sltu test
    lui x1, 0x80000
    addi x1, x1, 0x7ff
    lui x2, 0x80000
    addi x2, x2, 0x800
    sltu x18, x1, x2
    sltu x19, x2, x1

test_6:          # and test
    lui x1, 0xf0f0f
    addi x1, x1, 0xf0f
    lui x2, 0x0f0f0
    addi x2, x2, 0xf0f
    and x20, x1, x2

test_7:          # or test
    lui x1, 0xf0f0f
    addi x1, x1, 0xf0f
    lui x2, 0x0f0f0
    addi x2, x2, 0xf0f
    or x21, x1, x2

```

```

test_8:          # xor test
    lui x1, 0xf0f0f
    addi x1, x1, 0xf0f
    lui x2, 0x0f0f0
    addi x2, x2, 0xf0f
    xor x22, x1, x2

test_9:          # sll test
    addi x1, x0, 0x7ff
    addi x2, x0, 0x4
    sll x23, x1, x2

test_10:         # srl test
    addi x1, x0, 0x7ff
    addi x2, x0, 0x4
    srl x24, x1, x2

test_11:         # sra test
    lui x1, 0x80000
    addi x1, x1, 0x7ff
    addi x2, x0, 0x4
    sra x25, x1, x2

test_12:         # lb test
    lui x1, 0x00001
    addi x1, x1, 0x230
    lb x26, 0(x1)
    lb x27, 1(x1)
    lb x28, -1(x1)

test_13:         # lh test
    lui x1, 0x00001
    addi x1, x1, 0x230

```

```

        lh x29, 0(x1)
        lh x30, 2(x1)

test_14:        # lw test
        lui x1, 0x00001
        addi x1, x1, 0x230
        lw x31, 0(x1)

test_15:        # lbu test
        lui x1, 0x00001
        addi x1, x1, 0x230
        lbu x5, 0(x1)
        lbu x6, 1(x1)

test_16:        # lhu test
        lui x1, 0x00001
        addi x1, x1, 0x230
        lhu x7, 0(x1)

test_17:        # sb test
        lui x1, 0x00001
        addi x1, x1, 0x400
        addi x2, x0, 0xab
        sb x2, 0(x1)
        lb x8, 0(x1)

test_18:        # sh test
        lui x1, 0x00001
        addi x1, x1, 0x400
        lui x2, 0x00001
        addi x2, x2, 0xcdef
        sh x2, 0(x1)
        lh x9, 0(x1)

```

```

test_19:          # sw test
    lui x1, 0x00001
    addi x1, x1, 0x400
    lui x2, 0x12345
    addi x2, x2, 0x678
    sw x2, 0(x1)
    lw x10, 0(x1)

test_20:          # beq test
    addi x1, x0, 0x123
    addi x2, x0, 0x123
    beq x1, x2, label1
    addi x11, x0, 0xdead
label1:
    addi x11, x0, 0xbeef

test_21:          # bne test
    addi x1, x0, 0x123
    addi x2, x0, 0x456
    bne x1, x2, label2
    addi x12, x0, 0xdead
label2:
    addi x12, x0, 0xbeef

test_22:          # blt test
    addi x1, x0, 0x123
    addi x2, x0, 0x456
    blt x1, x2, label3
    addi x13, x0, 0xdead
label3:
    addi x13, x0, 0xbeef

```

```

test_23:          # bge test
    addi x1, x0, 0x456
    addi x2, x0, 0x123
    bge x1, x2, label4
    addi x14, x0, 0xdead
label4:
    addi x14, x0, 0xbeef

test_24:          # bltu test
    addi x1, x0, 0x123
    addi x2, x0, 0x456
    bltu x1, x2, label5
    addi x15, x0, 0xdead
label5:
    addi x15, x0, 0xbeef

test_25:          # bgeu test
    addi x1, x0, 0x456
    addi x2, x0, 0x123
    bgeu x1, x2, label6
    addi x16, x0, 0xdead
label6:
    addi x16, x0, 0xbeef

test_26:          # jal test
    jal x17, label7
    addi x18, x0, 0xdead
label7:
    addi x18, x0, 0xbeef

test_27:          # jalr test
    lui x1, 0x00001
    addi x1, x1, label8

```

```

        jalr x19, x1, 0
        addi x20, x0, 0xdead
label8:
        addi x20, x0, 0xbeef

test_28:        # auipc test
        auipc x21, 0x12345

test_29:        # lui test
        lui x22, 0x12345

test_30:        # addi test
        addi x23, x0, 0x7ff

test_31:        # slti test
        addi x1, x0, 0x7ff
        slti x24, x1, 0x800
        slti x25, x1, 0x7ff

test_32:        # sltiu test
        addi x1, x0, 0x7ff
        sltiu x26, x1, 0x800
        sltiu x27, x1, 0x7ff

test_33:        # xori test
        addi x1, x0, 0x7ff
        xori x28, x1, 0xff

test_34:        # ori test
        addi x1, x0, 0x7ff
        ori x29, x1, 0xff

test_35:        # andi test

```

```

        addi x1, x0, 0x7ff
        andi x30, x1, 0xff

test_36:        # slli test
        addi x1, x0, 0x7ff
        slli x31, x1, 0x4

test_37:        # srli test
        addi x1, x0, 0x7ff
        srli x5, x1, 0x4

test_38:        # srai test
        lui x1, 0x80000
        addi x1, x1, 0x7ff
        srai x6, x1, 0x4

test_39:        # sub test
        lui x1, 0x80000
        lui x2, 0xffff8
        sub x7, x1, x2

pass:
        lui x10, 0xc10
        addi x10, x10, -18      # R[10]=0x00c0ffee
        ecall                  # 程序正常结束

fail:
        lui x10, 0xdeadc
        addi x10, x10, -339     # R[10]=0xdeaddead
        ecall                  # 程序异常结束

```

2.5 错误现象及分析

一开始在设计 6.1 控制器的过程中，对于 ALUctr 最高位的输出不完备，没有考虑到 I 型指令中立即数占据了 funct7[5] 位的情况，导致在执行 0x7ff00013 指令时，ALUctr 错误输出为 1000，而不是 0000。后加入了特判，当 op 为 0x13 且 funct3 为 101，funct7[5] 为 1 时，对应的 I 型指令的 ALUctr 最高位输出为 0，解决了这个问题。

3 实验三 累加和程序测试实验

（由于本实验为测试实验，所以没有原理图和电路图）

3.1 测试用汇编程序

伪代码：

```
S=0;
for (i=1; i<=n; i++) S=S+i;
```

汇编语言如下：

#计算累加和程序

```
.text
main:
    lw a1,0(x0) # R[a1]:n,R[a1]<- Mem[0], 读取参数 n
    beq a1,x0,fail # if n=0 goto fail
    ori a2,x0,1 # R[a2]:i,=1
    xor a3,a3,a3 # R[a3]:S,=0
loop:
    add a3, a3, a2 # R[a3]=R[a3]+R[a2]
    beq a2, a1, finish # if R[a2]=n goto finish
11
    addi a2, a2, 1 # R[a2]=R[a2]+1
    jal x0, loop #
finish:
    sw a3, 4(x0) # Store S to Mem[4],Mem[4]<-R[a3]
```


pass:

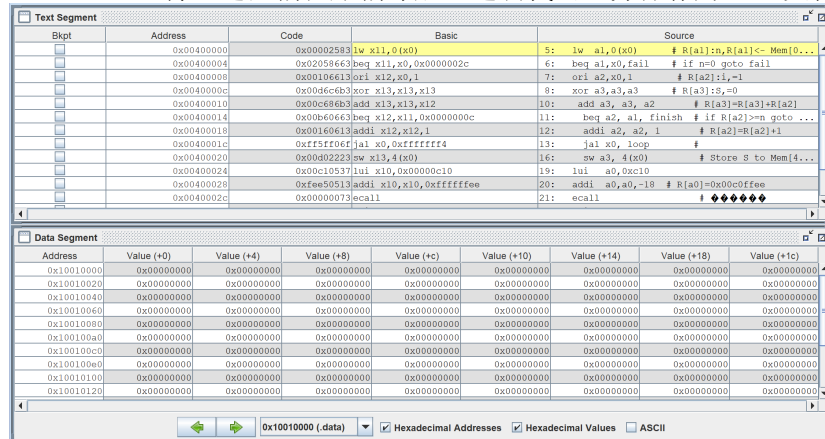
```
lui a0,0xc10
addi a0,a0,-18 # R[a0]=0x00c0ffee
ecall # 结束执行
```

fail:

```
lui a0,0xdead
addi a0,a0,-339 # R[a0]=0xdeaddead
ecall # 结束执行
```

3.2 将 RV32I 源程序汇编成二进制代码

通过 RARS 将上述汇编程序编译成二进制代码，并保存为 hex 文件。



3.3 加载测试程序

将 hex 文件加载到单周期 CPU 的指令存储器中，并将数据存储器地址 0 处初始化为测试参数 n。

经过测试，在 RAM0 输入 0x64 后，正确得到 0x13ba 的结果。

3.4 错误现象及分析

在完成本次实验的过程中，没有遇到任何错误。

4 冒泡排序程序测试实验

4.1 汇编代码

伪代码如下：

```
for (i=n; i>1; i--) {
    for (j=1; j<=i-1; j++) {
        if(a[j] >a[j+1]){
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}
```

对应汇编代码如下：

#冒泡排序算法

.text

main:

```
    lw t0,0(x0) # R[t0]<-Mem[0],t0 保存排序数量 n,
                # 待排序的数字个数 n 存在 0x00 处
```

```
    addi a1,x0,1 # a1,保存常量 1
```

```
    add a2,t0,x0 # a2,保存 i,初始值为 i=n
```

L1:

```
    addi a3,x0,1 # a3,保存 j,初始值为 j=1
```

L2:

```
    slli a4,a3,2 # a4 保存 a[j] 地址
```

```
    lw a6,0(a4) # 读取第 j 个元素
```

```
    lw a7,4(a4) # 读取第 j+1 个元素
```

```
    bge a7,a6,L4 # a[j]>=a[j+1] 跳转,按照带符号数比较
```

```
    sw a7,0(a4) # 交换存储
```

```
    sw a6,4(a4) # 交换存储
```

L4:

```
    addi a3,a3,1 # j=j+1
```

```

        bltu a3,a2,L2 # if j<i then 循环, 序号按照无符号数比较
L3:
        sub a2,a2,a1 # i--
        bne a2,a1,L1 # if i>1 then 循环 else 则结束
        ecall # 结束执行

```

4.2 将 RV32I 源程序汇编成二进制代码

通过 RARS 将上述汇编程序编译成二进制代码, 并保存为 hex 文件。

Offset	Address	Code	Basic	Source
0x00000000	0x00002283	lw a5,0(a0)	4:	lw a5,0(a0) # R[a0]←Mem[a0], 0保存排序数量, 待排序的数字个数=存在0x00处
0x00000004	0x00100593	addi a1,a0,1	5:	addi a1,a0,1 # a1, 保存常量1
0x00000008	0x00028633	addi a2,a0,0	6:	addi a2,a0,0 # a2, 保存i, 初始值为i=a
0x0000000c	0x00100693	addi a3,a0,1	7:	addi a3,a0,1 # a3, 保存j, 初始值为j=1
0x00000010	0x0009713	alli a4,a3,2	10:	alli a4,a3,2 # a4保存a[j]地址
0x00000014	0x00072803	lw a6,0(a4)	11:	lw a6,0(a4) # 读取a[j]个元素
0x00000018	0x00472803	lw a7,4(a4)	12:	lw a7,4(a4) # 读取a[j+1]个元素
0x0000001c	0x01084663	hgt a7,a6,a4	13:	hgt a7,a6,a4 # a[j]>a[j+1] 跳转, 按照无符号数比较
0x00000020	0x01172023	sw a7,0(a4)	14:	sw a7,0(a4) # 交换存储
0x00000024	0x01072223	sw a6,4(a4)	15:	sw a6,4(a4) # 交换存储
0x00000028	0x00188693	addi a3,a3,1	17:	addi a3,a3,1 # j=j+1
0x0000002c	0x00e6a2a3	bltu a3,a2,L2	18:	bltu a3,a2,L2 # if j<i then 循环, 序号按照无符号数比较
0x00000030	0x00469633	sub a2,a2,a1	20:	sub a2,a2,a1 # i--
0x00000034	0x00f461ca	bne a2,a1,L1	21:	bne a2,a1,L1 # if i>1 then 循环 else 则结束
0x00000038	0x00000073	ecall	22:	ecall # 结束执行

4.3 加载测试

在 Logisim 中打开单周期 CPU 电路图 lab6.2.circ, 在指令存储器中加载冒泡程序可执行机器代码数据镜像文件 lab6.4.hex, 在数据存储器 RAM0-RAM3 中分别加载待排序数据镜像文件 lab6.4_d.hex0-lab6.4_d.hex3。在 Logisim 的仿真菜单下, 选中时钟连续, CPU 开始自动执行机器代码。直到执行 ecall 指令, 此时按 Ctrl+K 终止“时钟连续”执行方式。

经过测试符合预期。

4.4 错误现象及分析

在完成本次实验的过程中, 没有遇到任何错误。

5 官方测试集实验

使用 testcase 中的测试文件, 依次测试所有 RV32I 指令, 并记录每条指令的测试结果, 包括 x1 寄存器输出值、x3 寄存器输出值、x10 寄存器输出值以及时钟周期数。

表 3: RISC-V 官方测试集测试数据表

序号	测试指令	x1 寄存器输出值	x3 寄存器输出值	x10 寄存器输出值	时钟周期数
1	add	0x0010	0x0026	0xc0ffee	458
2	addi	0x0021	0x0019	0xc0ffee	235
3	and	0x11111111	0x001b	0xc0ffee	478
4	andi	0x00ff00ff	0x000e	0xc0ffee	191
5	auipc	0	0x0003	0xc0ffee	52
6	beq	0x0003	0x0015	0xc0ffee	284
7	bge	0x0003	0x0018	0xc0ffee	302
8	bgeu	0x0003	0x0018	0xc0ffee	327
9	blt	0x0003	0x0015	0xc0ffee	284
10	bltu	0x0003	0x00015	0xc0ffee	309
11	bne	0x0003	0x0015	0xc0ffee	284
12	jal	0x0003	0x0003	0xc0ffee	48
13	jalr	0	0x0007	0xc0ffee	108
14	lb	0x2000	0x0013	0xc0ffee	238
15	lbu	0x2000	0x0013	0xc0ffee	238
16	lh	0x2000	0x0002	0xc0ffee	43
17	lhu	0x2000	0x0002	0xc0ffee	48
18	lui	0xffff800	0x0006	0xc0ffee	58
19	lw	0x2000	0x0013	0xc0ffee	260
20	or	0x11111111	0x001b	0xc0ffee	481
21	ori	0x00ff00ff	0x000e	0xc0ffee	198
22	sb	0x0001	0x0017	0xc0ffee	423
23	sh	0x2000	0x0004	0xc0ffee	65
24	sll	0x0400	0x002b	0xc0ffee	486
25	slli	0x0021	0x0019	0xc0ffee	234
26	slt	0x0010	0x0026	0xc0ffee	452
27	slti	0x00ff00ff	0x0019	0xc0ffee	230
28	sltiu	0x00ff00ff	0x0019	0xc0ffee	230
29	sltu	0x0010	0x0026	0xc0ffee	452
					续下页

续表：RISCV 官方测试集测试数据表

序号	测试指令	x1 寄存器输出值	x3 寄存器输出值	x10 寄存器输出值	时钟周期数
30	sra	0x0400	0x002b	0xc0ffee	505
31	srai	0x0021	0x0019	0xc0ffee	249
32	srl	0x0400	0x002b	0xc0ffee	499
33	srli	0x0021	0x0019	0xc0ffee	243
34	sub	0x0010	0x0025	0xc0ffee	450
35	sw	0x12233001	0x0017	0xc0ffee	483
36	xor	0x11111111	0x001b	0xc0ffee	480
37	xori	0x00ff00ff	0x000e	0xc0ffee	200

6 计算机系统基础 PA 程序测试实验

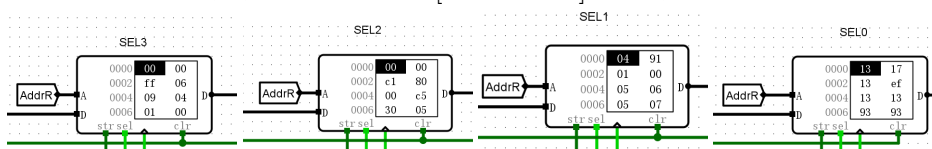
6.1 b-sort 程序测试

指令存储器加载：b-sort-riscv32-npc.bin-logisim-inst.txt，

数据存储器分别加载：b-sort-riscv32-npc.bin-logisim-data0.txt b-sort-riscv32-npc.bin-logisim-data3.txt

程序执行周期数：1348

程序执行结果，数据存储器地址 M[0x90-0xDC] 中的数据排列结果：



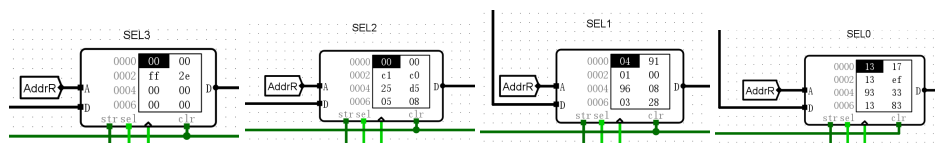
6.2 q-sort 程序测试

指令存储器加载：q-sort-riscv32-npc.bin-logisim-inst.txt，

数据存储器分别加载：q-sort-riscv32-npc.bin-logisim-data0.txt q-sort-riscv32-npc.bin-logisim-data3.txt

程序执行周期：999

程序执行结果，数据存储器地址 M[0x314-0x360] 中的数据排列结果：



7 思考题

7.1 如何在单 CPU 上实现多任务处理

可以采用任务切换的方式进行。在单周期 CPU 上，可以采用一个简单的任务列表，每个任务保存其程序计数器、寄存器状态和内存状态等信息。定义一个时钟中断，定期触发任务切换（比如每执行几条指令后触发一次任务切换）。在时钟中断处理程序中，保存当前任务的状态（如 PC、寄存器值）到任务列表中，然后从任务列表中选择下一个任务的状态进行恢复。当中断发生时，当前任务的 PC、寄存器值需要保存到对应的任务控制块中。需要恢复任务状态时，从任务列表中选择下一个任务，将其 PC 和寄存器值恢复到 CPU 中。

以同时计算累加和与数据排序为例，在初始化时，将计算累加和数据排序的任务分别加载到不同的任务控制块中，并初始化任务列表，然后在程序执行过程中通过时钟中断进行定期切换，使得计算累加和数据排序任务交替执行。

7.2 在 CPU 的基础上，如何实现键盘输入、TTY 输出部件等输入输出设备的数据访问，构建完整的计算机系统。

通过在 CPU 中增加存储器地址寄存器（MAR，存储当前访问的内存地址）和存储器数据寄存器（MDR，存储当前访问的内存数据），并使用内存映射 I/O 技术（将输入输出缓冲区映射到 CPU 的内存地址空间，使得 CPU 可以通过读写特定的内存地址来访问这些缓冲区），建立 CPU 和输入输出设备之间的联系。需要修改 CPU 控制器，扩展指令集，增加 IN 指令：从指定的 I/O 地址读取数据到 MDR，然后将数据传送到指定的寄存器；OUT 指令：从指定的寄存器读取数据到 MDR，然后将数据写入指定的 I/O 地址。

键盘输入的数据被转换为二进制存储到缓冲区，CPU 通过 IN 指令读取缓冲区中的数据。TTY 输出数据则通过 OUT 指令将 CPU 寄存器中的

数据写入缓冲区，再由 TTY 控制器输出到显示器。

7.3 阐述如何在单周期 CPU 基础上实现多周期 CPU

划分指令执行步骤：将指令划分为取指令，指令译码，执行，内存访问，写回五个阶段。

增加状态机控制器：使用一个状态机控制器，根据当前指令和状态，控制每个周期执行的步骤。状态机根据时钟信号转移到下一个状态。

修改控制逻辑：每个指令的操作被分解到不同的时钟周期中，通过状态机控制。在每个时钟周期执行不同的操作。

修改寄存器和数据通路：多周期 CPU 需要多个寄存器来保存每个步骤的中间结果。例如，IR（指令寄存器）等。

7.4 阐述如何在单周期 CPU 基础上实现流水线 CPU

分解指令执行阶段：取指令，指令译码，执行，内存访问，写回五个流水段

增加流水线寄存器：在每个阶段之间设置流水线寄存器，用于保存每个指令在不同阶段的中间结果。例如：IF/ID 寄存器、ID/EX 寄存器、EX/MEM 寄存器、MEM/WB 寄存器。

修改数据通路和控制逻辑：修改数据通路以支持流水线操作。

添加控制逻辑以处理数据冒险和控制冒险。